Quant Finance Starter Projects

Three small, self-contained projects that teach the core ideas of quantitative finance. Each is a single Python script you can read top to bottom.

```
pip install numpy pandas yfinance   # yfinance optional
python 01_monte_carlo_var.py        # real data via yfinance
python 02_pairs_trading.py --a KO --b PEP
python 03_ma_crossover.py --ticker SPY --cost-bps 10
# add --synthetic to any script to run offline on simulated data
```

If `yfinance` is missing or offline, the scripts fall back to synthetic data automatically.

---

1. Monte Carlo portfolio risk (`01_monte_carlo_var.py`)

Goal: How much could this portfolio lose in a bad month, and how bad does it get beyond that?

Steps

1. Download prices and compute daily log returns.
2. Estimate each stock's mean return and volatility, plus the correlation matrix.
3. Build the covariance matrix and take its **Cholesky decomposition** `L`, so that `L @ z` turns independent normal shocks `z` into correlated ones.
4. Simulate 10,000 paths of 21 trading days and compound each one into a one-month portfolio return.
5. **Value at Risk (95%)** = the 5th-percentile loss.
Expected Shortfall = the average loss in the worst 5% of outcomes.
6. Re-run with correlation forced to 0 and to 1.

What you learn: The volatilities stay the same in all three runs. Only correlation changes, and VaR moves a lot (in the sample run: $63k with zero correlation, $103k with the estimated correlation, $129k with perfect correlation). Diversification comes from correlation.

Extensions: fat tails (Student-t shocks), historical bootstrap instead of normal shocks, backtest VaR by counting breaches, optimise weights to minimise ES.

---

2. Pairs trading / statistical arbitrage (`02_pairs_trading.py`)

Goal: Profit when two related stocks drift apart, then come back together.

Steps

1. Split the history into a **formation** period (60%) and a **trading** period (40%).
2. On formation data, regress `A = alpha + beta * B`. `beta` is the **hedge ratio**.

3. **Engle-Granger test:** run an ADF test on the regression residuals. A statistic below about -3.34 means the pair is cointegrated at 5%. (These critical values are stricter than ordinary ADF ones because beta was estimated.)
4. Spread = `A - beta*B - alpha`. Z-score = (spread - rolling mean) / rolling std.
5. If z > +2, short the spread (short A, long B). If z < -2, go long the spread. Exit when z crosses 0.
6. Lag positions by one day and charge costs on both legs.
7. Compare in-sample and out-of-sample results, and re-test cointegration on the new data.

What you learn: Mean reversion and hedging. The biggest lesson is that **relationships break**. A pair that passes the test in-sample can stop being cointegrated later, so always validate out of sample and use a stop-loss (for example, exit at |z| > 4).

Extensions: rolling or Kalman-filter hedge ratio, half-life of mean reversion (fit an AR(1) to the spread), scanning a sector for candidate pairs (watch out for multiple-testing bias).

---

3. Moving-average crossover backtest (`03_ma_crossover.py`)

Goal: Test the simplest strategy there is, and test it properly.

Rules: Hold the asset when the 50-day SMA is above the 200-day SMA. Otherwise hold cash.

Testing discipline

• **Delay signals by one day.** The signal is computed on today's close, so you can only trade on it tomorrow (`position = signal.shift(1)`). The script also prints a look-ahead version that skips this lag. The gap between the two is the kind of performance a bug creates.
• **Transaction costs** are charged on every position change (default 10 bps).
• **Benchmark against buy and hold**, comparing CAGR, volatility, Sharpe ratio and max drawdown.

What you learn: Most strategies fail at this stage. The crossover usually gives up return in exchange for a smaller drawdown. If a backtest looks too good, look for look-ahead bias, survivorship bias, missing costs, or parameters fitted to the same data you tested on.

Extensions: a parameter sweep (and then ask why the best parameters won't hold up), walk-forward testing, applying it to many assets.

Checklist for any backtest

- ■ No future data: all signals lagged
- ■ Costs and slippage included
- ■ Benchmark comparison
- ■ Out-of-sample or walk-forward period
- ■ Parameters chosen before seeing test data
- ■ Survivorship-bias-free universe

Educational code, not investment advice.